Phone Dialplan Guide
Nuts and Bolts

How Asterisk and AllStarLink 3 route a phone call, line by line: the stock ASL3 dialplan, the one the Node Controller Dashboard writes for its Phone tab, and why each piece is there.

DIALPLAN APP_RPT AUTOPATCH SIP / IAX2 PHONE TAB

COVERS

TOOL	FILE	VERSION
Dashboard Pi02w	`nodecontrol_pi02w_v9_3_71_17_20261013.py`	9.3.71.17-pi02w
SysMon Pi02w	`sysmon_pi02w_v6_13_67_14_20261013.py`	6.13.67.14-pi02w
Dashboard (full)	`nodecontrol_v9_3_77_20261012.py`	9.3.77
SysMon (full)	`sysmon_v6_13_80_20261013.py`	6.13.80
Stock reference	AllStarLink app_rpt sample `rpt.conf` / `extensions.conf`, and its source (master `d392975`)	ASL3

In this guide

1. **The big picture**: the three parts of a phone call and how they meet
2. **Where the files live**: load order, includes, reloads and who owns what
3. **Anatomy of a dialplan line**: contexts, extensions, steps, labels
4. **Patterns**: how Asterisk decides which line matches a number
5. **Variables, functions and tests**: `${EXTEN}`, `CALLERID`, `DB()`, `$[ ]` and `GotoIf`
6. **The applications**: every app this dialplan uses, argument by argument
7. **Channels and calls**: PJSIP, IAX2, Local, bridges and Originate
8. **The app_rpt side**: node settings, function codes, `rpt()` options, duplex and VOX
9. **Inside autopatch**: call states, digit collection and the dead key
10. **The stock ASL3 dialplan**: what ships, and how you would turn it on
11. **The Node Controller dialplan**, context by context
12. **The phone nodes in rpt.conf**, and the three helper scripts
13. **The SIP account file**
14. **Call flows**: dashboard dial, radio `*61`, incoming, tones, hang-up
15. **Switches and the Dashboard's part**: AstDB keys, Originate, the dial poll
16. **Stock and Node Controller side by side**
17. **Hands-on**: watching a call, commands, common problems
18. **Quick reference**

How to read the examples

Config and dialplan lines are shown in green. Text after a `;` is a comment (grey): Asterisk ignores it. Commands you type on the Pi start with `sudo`. Numbers such as `555-234-5678` and passwords are examples.

> **NOTE** Every Node Controller excerpt in this guide was generated by the current Dashboard code for two example networks: **Hams Over IP** on node 1001 (extension dialing, incoming open) and **VoIP.ms** on node 1002 (phone numbers, E911, PIN on incoming, one trusted number). Your own files have the same shape with your names, numbers and nodes. Both builds write exactly the same files.

> **WORKING** The phone system works and has been used on a live node.

> **TESTING** **Hams Over IP**, the first example network, has been used to make calls but is still being tested.

> **TIP** You don't need to edit any of this by hand. The Dashboard rewrites its files on every Save Phone. This guide is for understanding what you see in SysMon's Phone tab, in the files, and in Asterisk's log.

The big picture

A phone call on an AllStar node is handled by three parts of the same Asterisk program. Each has its own config file and its own job:

PART	CONFIG	JOB
The dialplan	`extensions.conf`	Decides what happens to every call: which number goes where, what is checked first, when to answer, ring or hang up.
app_rpt (AllStar)	`rpt.conf`	Runs the nodes: radio audio, links between nodes, DTMF codes, keying the transmitter. Its `rpt()` app joins a call to a node; its autopatch places calls from the radio.
The channel drivers	`pjsip.conf`, `iax.conf`	Talk to the outside world: SIP providers (VoIP.ms, Hams Over IP, AmateurWire) and IAX2 servers, including signing in.

A call always enters the dialplan somewhere (a *context*), follows its steps, and usually ends in one of two places: `Dial()` (ring the outside world) or `rpt()` (join an AllStar node). The radio is never connected to a phone directly; it is always the node in the middle.

1. **Radio**
   Your handheld or mobile, on the node's radio interface (SimpleUSB on node 652702 in the examples).

▼

2. **Radio node**
   app_rpt node 652702. Keys the transmitter when a linked node or call has audio for it.

▼

3. **Phone node**
   A private node (1001) with no radio of its own. Linked to the radio node while the Phone tab is open.

▼

4. **The call**
   Joined to the phone node with `rpt(1001,Pv)`, or (stock) carried by the node's autopatch.

▼

5. **Phone network**
   PJSIP or IAX2 channel to the provider, created by `Dial()`.

> **WHY** Node Controller keeps the phone on its own node so the radio node's settings (simplex, hang time, links to the DMR and M17 bridges) are never mixed with phone settings, and so each phone network can have its own codes and rules.

Where the files live

Everything is in `/etc/asterisk`. Asterisk reads each file when its module loads, and again when you reload that module.

FILE	READ BY	RELOAD WITH	OWNED BY
extensions.conf	pbx_config	`dialplan reload`	You (stock file). Node Controller adds one include line at the end.
dvs_phone_extensions.conf	pbx_config (included)	`dialplan reload`	The Dashboard. Rewritten on every Save Phone.
rpt.conf	app_rpt	`module reload app_rpt`	You. Node Controller writes only between its two marker lines, plus one line per phone node in `[nodes]`.
pjsip.conf + dvs_phone_pjsip.conf	res_pjsip	`module reload res_pjsip.so` (and `res_pjsip_outbound_registration.so`)	You / the Dashboard (included file).
iax.conf + dvs_phone_iax*.conf	chan_iax2	`iax2 reload`	You / the Dashboard (included files).
modules.conf	Asterisk at start	restart Asterisk	You. Node Controller adds `load =>` lines for the phone modules.

Includes

A config file can pull in another file. Node Controller uses the safe form, which is silently skipped if the file is missing:

EXTENSIONS.CONF

```
; last line of /etc/asterisk/extensions.conf
#tryinclude dvs_phone_extensions.conf
```

An included file behaves exactly as if its text were pasted at that spot. Its contexts become part of the one big dialplan, so a stock context and a Node Controller context can jump to each other with `Goto`. The SIP file is included at the end of `pjsip.conf` the same way; the IAX2 sign-in file goes right after `[general]` in `iax.conf`, where `register =>` lines must be.

The rpt.conf markers

The phone nodes are written into `rpt.conf` itself, because app_rpt needs every node in that one file. The Dashboard only ever touches the text between these two lines:

RPT.CONF

```
; >>> dvs-phone (managed by asl_dvs_dashboard - edits inside are overwritten) >>>
[1001](node-main)
 ...
; <<< dvs-phone <<<
```

It also adds each phone node to the `[nodes]` list (`1001 = radio@127.0.0.1/1001,NONE`) so the radio node can link to it over the Pi's own IAX2 port.

CAREFUL Edits you make inside the markers, or inside `dvs_phone_*.conf`, are lost on the next Save Phone. Put your own contexts and nodes outside them. The Dashboard refuses to save if `rpt.conf` already has a `[<node>]` or `[functions<node>]` section it didn't write. It doesn't check dialplan contexts, so don't give your own contexts names starting with `dvs-`.

The first time the Dashboard changes one of your own files (`rpt.conf`, `extensions.conf`, `pjsip.conf`, `iax.conf`, `modules.conf`), it keeps a copy next to it named `<file>.dvsphone.bak`.

When changes take effect

- The Dashboard reloads what it changed after Save Phone: the dialplan, app_rpt, PJSIP or IAX2.
- New SIP modules need one Asterisk restart (tick **Restart Asterisk if needed** on the Edit page).
- About 20 seconds after the Dashboard starts, it compares what it would write with what is on disk and refreshes anything stale. That is how upgrades to the dialplan reach your Pi.

3 · SYNTAX

Anatomy of a dialplan line

The dialplan is a list of **contexts**. Each context is a list of **extensions**. Each extension is a numbered list of **steps** (Asterisk calls them priorities). Each step runs one **application**.

THE PARTS

```
[dvs-node-dvshoip]                          ; the context name, in square brackets
exten => _X.,1,Set(DVSNET=dvshoip)          ; extension _X., step 1, app Set
 same => n,Set(DVSOPT=)                     ; same extension, next step
 same => n(net),GotoIf(...)                 ; next step, with the label "net"
 same => n,Goto(dvs-net-dvshoip,${EXTEN},1) ; jump to another context
exten => i,1,Goto(dvs-invalid,s,1)          ; a second extension: "i" (invalid)
```

PIECE	MEANING
`[name]`	Starts a context. Everything below belongs to it until the next `[...]`.
`exten =>`	Starts an extension: `exten => what-was-dialed, step, App(arguments)`.
`_X.`	The extension. A leading `_` makes it a pattern (section 4); without it, it is matched literally, e.g. `911` or `s`.
`1`	The first step. Every extension must have a step 1.
`same =>`	Continue the extension above, so you don't repeat the pattern.
`n`	"Next": the step after the previous one. Asterisk numbers them 2, 3, 4... for you.
`n(net)`	Next step, and give it the label `net` so `Goto` / `GotoIf` can jump to it by name.
`App(args)`	The application and its arguments, separated by commas.
`; text`	A comment. Ignored by Asterisk.

How a call walks through it

1. The call arrives in a context with a number (the extension).

2. Asterisk finds the best matching `exten =>` line and runs its step 1.
3. It then runs step 2, 3… in order, unless a step jumps somewhere else (`Goto`, `GotoIf`).
4. An application such as `Dial` or `rpt` holds the call while it runs; when it returns, the next step runs.
5. Running off the end, or `Hangup()`, ends the call.

Jumps

FORM	GOES TO
`Goto(label)`	A label in the same extension.
`Goto(exten,step)`	Another extension in the same context.
`Goto(context,exten,step)`	Another context. Node Controller uses this to pass a call along: `Goto(dvs-net-dvshoip,${EXTEN},1)` keeps the dialed number.
`GotoIf($[test]?yes:no)`	Jump to *yes* if the test is true, *no* if false. Either side may be left out: `GotoIf($[test]?open)` simply carries on when false.

> **TIP** Read a context from the top of the matching extension downwards, and follow each `Goto`. That is exactly what SysMon's Dialing rules card does for you.

4 · SYNTAX

Patterns

An extension that starts with `_` is a pattern: it matches many numbers. Each character stands for one digit position, except `.` and `!`.

CHARACTER	MATCHES
`X`	Any digit 0–9.
`Z`	Any digit 1–9.
`N`	Any digit 2–9 (the first digit of a North American area code or exchange).
`[15-7]`	One digit from the set: here 1, 5, 6 or 7.
`.`	One or more of anything (digits, `*`, `#`).
`!`	Zero or more of anything, and match as soon as possible.
Any other character	Itself, e.g. `_1800NXXXXXX` needs a literal `1800`.

Patterns used in these dialplans

PATTERN	MATCHES	USED FOR
`_NXXNXXXXXX`	10 digits, valid area code and exchange	US/Canada number without the 1
`_1NXXNXXXXXX`	1 + 10 digits	the same number with the 1
`_NXXXXXX`	7 digits	stock: local number (adds your area code)

`_X11`	211, 311 … 911	stock: service codes
`_1800NXXXXXX`	toll-free 800 numbers	blocked list
`_1NXX555XXXX`	any 555 exchange	blocked list (directory and fictional numbers)
`_011.`	011 and more	international (only if allowed)
`_X.`	a digit then anything	"everything else": catch-alls and extension dialing
`_[+0-9].`	a digit or + then anything	incoming caller ID or DID numbers
`_XXXX!`	4 or more digits	stock: node numbers in `radio-secure`

NOTE The exchange (the 3 digits after the area code) must start with 2–9 too. A number like 555-**1**23-4567 matches neither `_NXXNXXXXXX` nor `_1NXXNXXXXXX`, and the Dashboard refuses it before dialing. That is why the examples here use 555-234-5678.

Which line wins

When several patterns match, Asterisk picks the most specific one: an exact number beats a pattern, and a pattern with fewer possibilities beats a looser one. So in the VoIP.ms rules below, `911` wins over `_X.` even though both match. In the next context, `dvs-net-dvsvoipms-r` (section 11), the blocked `_1800NXXXXXX` wins over the general `_1NXXNXXXXXX` the same way.

DVS_PHONE_EXTENSIONS.CONF

```
[dvs-net-dvsvoipms]
exten => 911,1,NoOp(911 - E911 is set up on this network)
 same => n,Set(CALLERID(num)=5550001111)
 same => n,Dial(PJSIP/911@dvsvoipms,60)
 same => n,Hangup()
exten => _NXXNXXXXXX,1,Goto(dvs-net-dvsvoipms-r,1${EXTEN},1)
exten => _1NXXNXXXXXX,1,Goto(dvs-net-dvsvoipms-r,${EXTEN},1)
exten => _X.,1,Goto(dvs-invalid,s,1)
exten => i,1,Goto(dvs-invalid,s,1)
```

Special extensions

EXTENSION	WHEN IT RUNS
`s`	"Start": a call that arrives with no number (many incoming SIP calls).
`i`	"Invalid": a number that matches no line in this context.
`t`	"Timeout": the caller didn't enter anything in time.
`h`	"Hang-up": runs after the call ends (not used here).

CAREFUL A single digit never matches `_X.`: the dot needs at least one more character. That is why the autopatch waits for at least two digits.

Variables, functions and tests

Anything inside `${ }` is replaced by its value before the step runs.

WRITTEN	VALUE
`${EXTEN}`	The number being handled, e.g. `15552345678`.
`${EXTEN:1}`	The same, minus its first character: `5552345678`. (`:2` drops two, and so on.)
`${CALLERID(num)}`	The caller's number.
`${CALLERID(num):-10}`	Its last 10 characters, so `+15553456789` and `15553456789` both become `5553456789`.
`${DB(dvsphone/active)}`	Reads a value from Asterisk's database (AstDB): family `dvsphone`, key `active`.
`${DB_EXISTS(dvsphone/then)}`	1 if the key exists, 0 if not.
`${DB_DELETE(dvsphone/then)}`	Returns the value and deletes the key, so it is used exactly once.
`${DVSNET}, ${DVSOPT}`	Variables set earlier in the same call with `Set()`.

Setting variables

EXAMPLES

```
 same => n,Set(DVSNET=dvshoip)                              ; a channel variable
 same => n,Set(CALLERID(num)=5550001111)                    ; a function can be written to
 same => n(then),Set(DVSOPT=D(${DB_DELETE(dvsphone/then)})) ; read-and-delete
```

Tests

`$[ ... ]` works out an expression: `=`, `!=`, `<`, `>`, `&` (and), `|` (or), `+ - * /`. When both sides are numbers they are compared as numbers, otherwise as text. A value that might be empty would leave a gap in the expression, so Node Controller puts quotes around such values:

TESTS USED HERE

```
; incoming: only if the Phone tab is open AND this network is the picked one
 same => n,GotoIf($["${DB(dvsphone/open)}" = "1" & "${DB(dvsphone/active)}" = "dvshoip"]?open)

; outgoing: stop if the phone patch is switched off
 same => n(net),GotoIf($["${DB(dvsphone/patch)}" = "0"]?bad)

; PIN: count the tries
 same => n,Set(TRIES=$[${TRIES} + 1])
 same => n,GotoIf($[${TRIES} >= 3]?bye)
```

WHY Reading switches from AstDB lets the dialplan react the moment you change something on the Phone tab, with no reload: the Dashboard writes `dvsphone/open` when you open the tab, `dvsphone/active` when you pick a network, and `dvsphone/patch` when you turn the patch on or off.

The applications

These are every application the stock and Node Controller phone dialplans use. Arguments are separated by commas; empty ones keep their default.

APPLICATION	ARGUMENTS AS USED HERE	WHAT IT DOES
`Answer()`	—	Picks up an incoming call. Audio can now flow both ways.
`Wait(1)`	seconds	Pauses. Used so a greeting isn't clipped right after answering.
`Playback(rpt/connected)`	sound file(s), joined with `&`	Plays recordings: `rpt/connected`, `ss-noservice`, `confbridge-invalid`, `confbridge-pin-bad`.
`Read(PIN,beep,8,,1,10)`	variable, prompt, max digits, options, attempts, timeout	Plays a beep and collects up to 8 digits (ended by #) into `PIN`, waiting 10 s.
`Dial(PJSIP/${EXTEN}@dvsvoipms,60,${DVSOPT})`	Tech/number@account, ring seconds, options	Rings the number through that account for up to 60 s. Option `D(digits)` sends tones after answer.
`Busy(20)`	seconds	Signals busy to the caller and waits up to 20 s for them to hang up. The `Hangup()` after it ends the call if they don't.
`Hangup()`	—	Ends the call.
`Goto / GotoIf`	see section 3	Jump to another step, extension or context.
`NoOp(text)`	any text	Does nothing; the text shows in the Asterisk log. Handy as a label for humans.
`Set(VAR=value)`	assignment	Sets a variable or writable function (section 5).
`System(command)`	a shell command	Runs a program on the Pi. Node Controller uses it once: to hand a radio-dialed number to the Dashboard.
`rpt(1001,Pv) / Rpt(...)`	node, options	Joins the call to an AllStar node (section 8).

Dial in detail

DIAL

```
Dial(PJSIP/${EXTEN}@dvsvoipms,60,D(ww*99))
;    \_____________________________/ \/ \________/
;     who to ring                  ring after answer: wait 1 s (each w = 0.5 s),
;     (technology/number@account) 60 s then send the tones *99
```

TECHNOLOGY STRING	MEANING
`PJSIP/15552345678@dvsvoipms`	SIP: send the number to the PJSIP endpoint `dvsvoipms` (section 13).
`PJSIP/*97@dvshoip`	SIP: an extension or feature code on Hams Over IP (here voicemail).
`IAX2/dvsclub/2001`	IAX2: ring 2001 through the IAX2 peer `dvsclub`.
`IAX2/allstar-autopatch/${EXTEN}`	Stock example: the AllStarLink autopatch service.

> **NOTE** Node Controller fills `D()` only for a favorite with saved tones, and only when **Send keypad tones** is set to **Through the call**. Otherwise the Dashboard sends the tones itself once the call is answered.

7 · CALLS

Channels and calls

Every leg of a call is a **channel**. A channel has a technology, a name, and a place in the dialplan. Two channels talking to each other are in a **bridge**.

CHANNEL NAME LOOKS LIKE	WHAT IT IS
`PJSIP/dvsvoipms-0000002a`	A SIP call leg to or from the VoIP.ms account.
`IAX2/dvsclub-1234`	An IAX2 call leg to or from an IAX2 phone network.
`Local/5552345678@dvs-node-dvsvoipms-00000003;1`	Half of a Local channel: a call Asterisk places to itself, into a context.
`...;2`	The other half of the same Local channel. It runs the context's steps (here, `Dial`).

Local channels

`Local/number@context` lets a program start a call into any context as if someone had dialed *number*. It always comes as a pair:

1. **Half ;1**
   Runs whatever the caller asked for: here the app `rpt(1002,Pv)`, once the call is answered.

▼

2. **Half ;2**
   Walks the context: `dvs-node-dvsvoipms` → `dvs-net-dvsvoipms` → `-r` → `Dial()`.

▼

3. **PJSIP leg**
   Created by `Dial()`. When the far end answers, ;2 answers, which answers ;1.

The `/n` at the end (`Local/...@dvs-node-dvsvoipms/n`) tells Asterisk not to optimise the pair away after answer, so the call keeps a stable shape the Dashboard can follow.

Originate: placing a call from a program

The Dashboard talks to Asterisk over the Asterisk Manager Interface (AMI). To place a call it sends:

AMI ORIGINATE (SENT BY THE DASHBOARD)

```
Action: Originate
Channel: Local/5552345678@dvs-node-dvsvoipms/n  ; dial this, through this context
Application: Rpt                                ; once answered, run this app ...
Data: 1002,Pv                                   ; ... with these arguments
Timeout: 65000                                  ; give up after 65 s of ringing
Async: true                                     ; don't wait; report back later
```

So a Dashboard call is really two things glued together: the ordinary dialing rules (on ;2) and `rpt(1002,Pv)` (on ;1). If the far end is busy or never answers, ;1 is never answered and `rpt()` never runs: nothing touches the radio.

TIP In `core show channels` during a Dashboard call you will see three lines: the PJSIP (or IAX2) leg running `AppDial`, `Local/...;2` running `Dial`, and `Local/...;1` running `Rpt`.

8 · ALLSTAR

The app_rpt side

Node settings that matter for phones

SETTING	MEANING
`context = dvs-node-dvshoip`	The node's default dialing context (used by autopatch when a code doesn't name one).
`functions = functions1001`	Codes keyed on the node's own radio (this node has none), and the default for links.
`link_functions = functions1001`	Codes arriving over a link, i.e. keyed on your radio node and passed along.
`phone_functions = functions1001`	Codes keyed by a caller joined with `rpt(node,P...)`.
`rxchannel = Local/pseudo`	No radio hardware: a software-only node.
`duplex, vox*, simplex*`	Keying behaviour (below).

Function codes

A function table maps a DTMF code (keyed after the node's `*`) to an action:

ACTION	EXAMPLE	DOES
`autopatchup`	`61 = autopatchup,noct=1,...,quiet=1`	Starts the autopatch and collects the number.
`autopatchdn`	`62 = autopatchdn`	Ends the autopatch (stock).
`cmd`	`62 = cmd,/var/lib/asterisk/dvs_phone_hangup`	Runs a program on the Pi.
`cop,6`	`99 = cop,6`	Phone-mode PTT: lets a phone caller key the radio with `*99`.
`cop,9` / `cop,10`	`63 = cop,9`	Autopatch enable / disable.
`ilink`	`3 = ilink,3`	Link commands on the radio node (3 connects two-way, 2 monitor, 1 disconnects).

autopatchup options

OPTION	MEANING
`context=name`	Dial through this context instead of the node's `context =`.
`dialtime=20000`	Give up if no digit arrives for this many ms.
`farenddisconnect=1`	End the autopatch when the far end (or the dialplan) hangs up.
`noct=1`	No courtesy tone during the call.
`quiet=1`	No dial tone or spoken messages.
`voxalways=1`	Use VOX on the patch audio even on a full-duplex node.
`exten=number`	Dial a fixed number instead of collecting digits.
`nostar`	Don't treat `*` as the start of a new command while dialing.

rpt() options

The first letter picks the mode, later letters modify it:

OPTION	MODE
`P`	Phone control. The caller has audio and DTMF control through `phone_functions`; PTT with the `cop,6` code (`*99`), unkey with #.
`D`	Dumb phone. Keys the transmitter for the whole call.
`S`	Dumb simplex. The caller toggles transmit with `*`; # turns it off.
`R`	Reverse autopatch. Parks the caller and announces the call on the radio.
`X`	Normal node link without the security check (stock `[iaxrpt]`).
add `v`	VOX: also key while the caller's voice is present. Node Controller uses `Pv` (section 9).
add `m`	Monitor: the phone gets no transmit or functions.

Duplex and VOX

DUPLEX	BEHAVIOUR
0	Half duplex, no telemetry or hang time. Your radio node 652702 uses this.
1	Half duplex with telemetry and hang time; doesn't repeat audio. The phone nodes use this.
2	Full duplex repeater (the ASL3 template default).
3	Full duplex, no repeated audio.
4	Full duplex, repeat only when autopatch is down.

VOX SETTING	DEFAULT	MEANING
`voxtimeout`	10000 ms	Longest the VOX may hold the transmitter.
`voxrecover`	2000 ms	Forced gap after a timeout, so the radio side can break in.

`simplexpatchdelay`	25 × 20 ms	Audio delay before transmitting patch audio, so the first word isn't clipped.
`simplexphonedelay`	25 × 20 ms	The same, for a phone-mode caller's audio.

9 · ALLSTAR

Inside autopatch

Autopatch is app_rpt's built-in phone patch. Knowing its states explains both how stock dialing works and why Node Controller stopped using it for the call itself.

STATE	WHAT IS HAPPENING
DOWN	No patch.
DIALING	`*61` was keyed; digits are being collected (dial tone unless `quiet=1`).
CONNECTING	The number is complete; the dialplan is running (`Dial()` is ringing).
UP	The call is connected.
FAILED	The number can't match anything in the context; congestion until the patch is taken down.

How it knows the number is complete

1. Each digit is added to the number and checked against the context's patterns.
2. If the number matches a line and no longer number could match, it dials at once.
3. If a longer number could still match (always the case with `_X.`), it waits 1.5 seconds after the last digit, then dials.
4. If no digit arrives for `dialtime` (20 s here), it gives up.

How it ends

With `farenddisconnect=1`, or on any half-duplex node, the patch goes down as soon as the dialplan finishes or the far end hangs up. Otherwise it waits for the `autopatchdn` code.

The dead key

While a patch is in DIALING, CONNECTING or UP, app_rpt keys every linked node for the whole call, whether or not anyone is talking. The code that would gate this by voice exists, but is compiled out because it clipped the first syllable on links:

APP_RPT.C, PER-LINK KEYING (MASTER D392975, LINES 4916–4921)

```
mycalltx = myrpt->callmode;
#ifdef DONT_USE__CAUSES_CLIPPING_OF_FIRST_SYLLABLE_ON_LINK
    if (myrpt->patchvoxalways)
        mycalltx = mycalltx && ((!myrpt->voxtostate) && myrpt->wasvox);
#endif
totx = ((l->isremote) ? (remnomute) : (myrpt->localtx && myrpt->totimer) || mycalltx) || remrx;
```

1 **Phone node 1001 is in autopatch**
`callmode` is not DOWN, so it keys its link for the whole call.

2. **Radio node 652702 sees the link keyed**
   It transmits the link audio, continuously.

3. **Your simplex radio can't receive**
   The hotspot is transmitting, so it never hears you key up.

4. **Result**
   Receive audio works, but nothing you say reaches the call. `*99` seems dead.

WHY A full-duplex repeater doesn't mind; a simplex hotspot can't work with it. Settings such as `duplex`, `vox*` or `voxalways` never reach the line above, so no `rpt.conf` setting can fix it.

What Node Controller does instead

Since 9.3.71.8-pi02w and 9.3.73 the call never lives in autopatch. Calls dialed from the Dashboard and incoming calls join the phone node with `rpt(<node>,Pv)`, phone mode with VOX: the same mode as the stock `_3.` example in `[allstar-sys]` (the stock `[iax-client]` uses plain `P`). `*61` from the radio still uses autopatch, but only to collect the digits: its context hands the number to the Dashboard and hangs up, so the patch drops at once and the Dashboard places the call the same `Pv` way. The link is keyed only for those few seconds of dialing.

How Pv keys

1. **The far end talks**
   Each 20 ms frame of the caller's audio goes through app_rpt's VOX detector. It turns on after 3 voiced frames and off after 20 quiet ones (about 0.4 s), so pauses between words don't drop it.

2. **The call counts as receiving**
   While VOX hears voice (or `*99` has it keyed), the phone node treats the call's link as receiving, and keys its other links: the one to your radio node.

3. **The first word survives**
   When VOX first triggers, `simplexphonedelay` (25 × 20 ms, about 0.5 s) of silence goes in front of the audio, so the transmitter is up before the words arrive.

4. **The far end stops**
   VOX drops, the link unkeys, your hotspot goes back to receive, and what you say goes into the call.

5. **Nobody holds it forever**
   After `voxtimeout` (10 s) of nonstop far-end audio, VOX is forced off for `voxrecover` (2 s) so the radio side can break in.

Source, app_rpt master d392975: option parsing app_rpt.c 7354–7367; link VOX and timeout 3567–3581; keying other links 4898–4906; first-word delay 5014–5030; VOX detector rpt_vox.c dovox(); defaults app_rpt.h 198–205.

The stock ASL3 dialplan

A fresh ASL3 install has phone calling turned off. The pieces are there, commented out or pointed at a "no service" message.

STOCK RPT.CONF (EXCERPT)

```
[node-main](!)
functions = functions          ; Repeater Function stanza
link_functions = functions     ; Link Function stanza
phone_functions = functions    ; Phone Function stanza
context = radio                ; dialing context for phone
duplex = 2

[functions-main](!)
99 = cop,6                     ; PTT (phone mode only)
;61 = autopatchup,noct = 1,farenddisconnect = 1,dialtime = 20000  ; Autopatch up
;62 = autopatchdn                                                 ; Autopatch down
```

STOCK EXTENSIONS.CONF (EXCERPT)

```
[radio]
exten => _X11,1,Goto(check_route,${EXTEN},1)
exten => _NXXXXXX,1,Goto(check_route,1${HOMENPA}${EXTEN},1)  ; 7 digits: add your area code
exten => _1XXXXXXXXXX,1,Goto(check_route,${EXTEN},1)
exten => _07XX,1,Goto(parkedcalls,${EXTEN:1},1)
exten => 00,1,Goto(my-ip,s,1)                                ; speaks the node's IP address

[check_route]
exten => _X.,1,NoOp(${EXTEN})
exten => _1800NXXXXXX,2,Goto(invalidnum,s,1)                 ; also 888 877 866 855
exten => _1X00XXXXXXX,2,Goto(invalidnum,s,1)                 ; X00 area codes
exten => _1X11XXXXXXX,2,Goto(invalidnum,s,1)
exten => _X11,2,Goto(invalidnum,s,1)                         ; 211 ... 911
exten => _1NXX555XXXX,2,Goto(invalidnum,s,1)
exten => _1NXX976XXXX,2,Goto(invalidnum,s,1)
exten => _1809XXXXXXX,2,Goto(invalidnum,s,1)
exten => _1900XXXXXXX,2,Goto(invalidnum,s,1)
exten => _1NXXXXXXXXX,2,Goto(pstn-out,${EXTEN:1},1)          ; okay, route it
exten => _X.,2,Goto(invalidnum,s,1)

; Comment-out the following clause if you want Allstar Autopatch service
[pstn-out]
exten => _NXXNXXXXXX,1,Wait(1)
 same => n,Playback(ss-noservice)
 same => n,Hangup

[invalidnum]
exten => s,1,Wait(3)
 same => n,Playback(ss-noservice)
 same => n,Wait(1)
 same => n,Hangup
```

Note the trick in `[check_route]`: every number first hits step 1 of `_X.`; step 2 is then taken from the most specific matching line, so the blocked kinds win over the final "route it" line. That includes `_X11`: stock sends 911 and the other service codes to "no service".

Other stock contexts

CONTEXT	PURPOSE
`[radio-secure]`	Node-to-node connections, with allow and deny lists in AstDB; ends in `Rpt(${EXTEN})`.
`[iaxrpt]`	IAX apps that log in with a user/password; `Rpt(${EXTEN},X)`.
`[iax-client]`	IAX VoIP clients: rings, answers, then `Rpt(${EXTEN},P,${CALLSIGN}-P)`: phone mode, keyed with `*99`.
`[allstar-sys]`	Dial-in examples for each `rpt()` mode, picked by the first digit: `_1.` reverse autopatch (`R`), `_2.` `P`, `_3.` `Pv`, `_4.` `D`, `_5.` `Dv`.
`[allstar-public]`	Web-phone connections, checked against AllStarLink's server and the allow and deny lists; `Rpt(${NODENUM},X)`.
`[my-ip]`, `[telemetry]`	Speaks the node's public IP address (dialed as `00`); telemetry helpers.

Turning stock autopatch on

1. Uncomment `61` and `62` in your node's function table.
2. In `[pstn-out]`, replace the "no service" lines with a `Dial()` to your provider (stock suggests `IAX2/allstar-autopatch`).
3. Set `HOMENPA` in `[globals]` to your area code, then reload app_rpt and the dialplan.

> **CAREFUL** Done on the radio node itself, a stock autopatch call keys that node and every node linked to it for the whole call (section 9): your DMR and M17 bridges would hear the phone too.

11 · NODE CONTROLLER

The Node Controller dialplan, context by context

The Dashboard writes one file, `dvs_phone_extensions.conf`. It starts with a few shared contexts, then a block of contexts per phone network.

CONTEXT	REACHED FROM	JOB
`dvs-invalid`	any "bad number" jump	"No service", then hang up.
`dvs-phone-out`	the Phone Bridge node's `context =`, when no network owns that node	Outgoing front door that reads the picked network from AstDB.
`dvs-radio-out`	`*61` on that Phone Bridge node	Hands the number to the Dashboard.
`dvs-tones`	the Dashboard only	Plays touch-tone recordings into a call ("As sound").
`dvs-node-<net>`	Originate; the node's `context =`	Outgoing front door for one network.
`dvs-radio-<net>`	`*61` on that network's node	Hands the number to the Dashboard.
`dvs-net-<net>`	`dvs-node-<net>`	The network's dialing rules.
`dvs-net-<net>-r`	`dvs-net-<net>` (phone numbers only)	Blocked list, then `Dial()`.

`dvs-in-<net>`	the SIP or IAX2 account	Incoming calls.

Shared contexts

DVS-INVALID

```
[dvs-invalid]
exten => s,1,Wait(1)
 same => n,Playback(ss-noservice)
 same => n,Hangup()
```

DVS-PHONE-OUT

```
[dvs-phone-out]
exten => _X.,1,Set(DVSNET=${DB(dvsphone/active)})
 same => n,Set(DVSOPT=)
 same => n,GotoIf($[${DB_EXISTS(dvsphone/then)}]?then:net)
 same => n(then),Set(DVSOPT=D(${DB_DELETE(dvsphone/then)}))
 same => n(net),GotoIf($["${DVSNET}" = ""]?bad)
 same => n,GotoIf($["${DB(dvsphone/patch)}" = "0"]?bad)
 same => n,Goto(dvs-net-${DVSNET},${EXTEN},1)
 same => n(bad),Goto(dvs-invalid,s,1)
exten => i,1,Goto(dvs-invalid,s,1)
exten => t,1,Hangup()
```

DVS-RADIO-OUT

```
[dvs-radio-out]
exten => _X.,1,NoOp(radio dial)
 same => n,System(/var/lib/asterisk/dvs_phone_dialreq "${EXTEN}" "${DB(dvsphone/active)}")
 same => n,Hangup()
exten => i,1,Hangup()
exten => t,1,Hangup()
```

DVS-TONES

```
[dvs-tones]
exten => s,1,Answer()
 same => n,Wait(0.5)
 same => n,Playback(${DVSTONES})
 same => n,Wait(0.3)
 same => n,Hangup()
```

`dvs-tones` is started by the Dashboard with Originate and whispered into the live call with ChanSpy, when **Send keypad tones** is set to **As sound**. `${DVSTONES}` is the list of digit recordings to play.

The outgoing front door

DVS-NODE-DVSHOIP

```
[dvs-node-dvshoip]
exten => _X.,1,Set(DVSNET=dvshoip)
 same => n,Set(DVSOPT=)
 same => n,GotoIf($[${DB_EXISTS(dvsphone/then)}]?then:net)
 same => n(then),Set(DVSOPT=D(${DB_DELETE(dvsphone/then)}))
 same => n(net),GotoIf($["${DB(dvsphone/patch)}" = "0"]?bad)
 same => n,Goto(dvs-net-dvshoip,${EXTEN},1)
 same => n(bad),Goto(dvs-invalid,s,1)
exten => i,1,Goto(dvs-invalid,s,1)
exten => t,1,Hangup()
```

STEP	WHY
`Set(DVSNET=dvshoip)`	Records which network this is (for the log and later steps).
`Set(DVSOPT=)`	Starts with no Dial options.
`GotoIf(DB_EXISTS(dvsphone/then)...)`	A favorite with saved tones is waiting: put `D(...)` in `DVSOPT` and delete the key so it is used once.
`GotoIf(... patch = 0 ...?bad)`	The Phone patch switch is off: refuse.
`Goto(dvs-net-dvshoip,${EXTEN},1)`	On to the network's rules, keeping the number.
`exten => i, t`	Anything strange: "no service" or hang up.

The radio hand-off

DVS-RADIO-DVSHOIP

```
[dvs-radio-dvshoip]
exten => _X.,1,NoOp(radio dial)
 same => n,System(/var/lib/asterisk/dvs_phone_dialreq "${EXTEN}" dvshoip)
 same => n,Hangup()
exten => i,1,Hangup()
exten => t,1,Hangup()
```

The number is passed in quotes, so `*` or `#` can't be read by the shell as anything else; the script then refuses anything that isn't plain digits. `Hangup()` ends the autopatch straight away (section 9), so the radio is never keyed by the call.

Dialing rules: an extension network (Hams Over IP)

DVS-NET-DVSHOIP

```
[dvs-net-dvshoip]
exten => 0098,1,NoOp(test call)
 same => n,Dial(PJSIP/3191@dvshoip,60,${DVSOPT})
 same => n,Hangup()
exten => 0097,1,NoOp(voicemail)
 same => n,Dial(PJSIP/*97@dvshoip,60,${DVSOPT})
 same => n,Hangup()
exten => _X.,1,NoOp(extension call)
 same => n,Dial(PJSIP/${EXTEN}@dvshoip,60,${DVSOPT})
 same => n,Hangup()
exten => i,1,Goto(dvs-invalid,s,1)
```

LINE	MEANING
`0098`	The Phone tab's Test call row: dials the network's saved test number (3191 here, the Hams Over IP tone read-back).
`0097`	The Voicemail row: dials the saved voicemail access code (`*97` here). Only for Hams Over IP and AmateurWire.
`_X.`	Any extension you dial: sent as typed.
`i`	Anything else: "no service".

Dialing rules: a phone-number network (VoIP.ms)

DVS-NET-DVSVOIPMS

```
[dvs-net-dvsvoipms]
exten => 911,1,NoOp(911 - E911 is set up on this network)
 same => n,Set(CALLERID(num)=5550001111)
 same => n,Dial(PJSIP/911@dvsvoipms,60)
 same => n,Hangup()
exten => _NXXNXXXXXX,1,Goto(dvs-net-dvsvoipms-r,1${EXTEN},1)
exten => _1NXXNXXXXXX,1,Goto(dvs-net-dvsvoipms-r,${EXTEN},1)
exten => _X.,1,Goto(dvs-invalid,s,1)
exten => i,1,Goto(dvs-invalid,s,1)
```

- `911` exists only because **E911 is set up** is ticked for this network. It skips the blocked list and the saved-tones option.
- 10 digits get a leading 1, then both forms go to the `-r` context.
- `Set(CALLERID(num)=...)` sends your caller ID number, if you filled one in.
- With **international** allowed, an `_011.` line is added that dials as typed.
- A **Test call** number, if set, adds a `0098` line here too.
- The number format the provider wants (10 digits, 1+10, +1+10) is chosen per network on the Edit page and decides the `${EXTEN:1}` / `${EXTEN}` / `+${EXTEN}` in `Dial`. The example uses 1+10.

DVS-NET-DVSVOIPMS-R

```
[dvs-net-dvsvoipms-r]
exten => _1800NXXXXXX,1,Goto(dvs-invalid,s,1)
exten => _1888NXXXXXX,1,Goto(dvs-invalid,s,1)
exten => _1877NXXXXXX,1,Goto(dvs-invalid,s,1)
exten => _1866NXXXXXX,1,Goto(dvs-invalid,s,1)
exten => _1855NXXXXXX,1,Goto(dvs-invalid,s,1)
exten => _1N00XXXXXXX,1,Goto(dvs-invalid,s,1)
exten => _1N11XXXXXXX,1,Goto(dvs-invalid,s,1)
exten => _1NXX555XXXX,1,Goto(dvs-invalid,s,1)
exten => _1NXX976XXXX,1,Goto(dvs-invalid,s,1)
exten => _1809XXXXXXX,1,Goto(dvs-invalid,s,1)
exten => _1900XXXXXXX,1,Goto(dvs-invalid,s,1)
exten => _1NXXNXXXXXX,1,NoOp(dialing)
 same => n,Set(CALLERID(num)=5550001111)
 same => n,Dial(PJSIP/${EXTEN}@dvsvoipms,60,${DVSOPT})
 same => n,Hangup()
```

The blocked kinds follow stock `check_route` (with `N` instead of `X` for the first digit of the area code). They win over the general `_1NXXNXXXXXX` line because they are more specific (section 4). The service codes (`_X11`) never get here: three digits fall to `_X.` in the context above, unless the `911` line exists.

Incoming: open (Hams Over IP)

DVS-IN-DVSHOIP

```
[dvs-in-dvshoip]
exten => _[+0-9].,1,NoOp(incoming call from ${CALLERID(num)})
 same => n,GotoIf($["${DB(dvsphone/open)}" = "1" & "${DB(dvsphone/active)}" = "dvshoip"]?open)
 same => n,Busy(20)
 same => n,Hangup()
 same => n(open),NoOp()
 same => n(connect),Answer()
 same => n,Wait(1)
 same => n,Playback(rpt/connected)
 same => n,rpt(1001,Pv)
 same => n,Hangup()
exten => s,1,NoOp(incoming call from ${CALLERID(num)})
 same => n,GotoIf($["${DB(dvsphone/open)}" = "1" & "${DB(dvsphone/active)}" = "dvshoip"]?open)
 same => n,Busy(20)
 same => n,Hangup()
 same => n(open),NoOp()
 same => n(connect),Answer()
 same => n,Wait(1)
 same => n,Playback(rpt/connected)
 same => n,rpt(1001,Pv)
 same => n,Hangup()
```

The `s` extension repeats the same steps, for calls that arrive with no number. With incoming **open** and trusted numbers listed, only the trusted numbers get through; everyone else gets busy.

Incoming: PIN and trusted numbers (VoIP.ms)

DVS-IN-DVSVOIPMS (THE S EXTENSION REPEATS THE SAME STEPS)

```
[dvs-in-dvsvoipms]
exten => _[+0-9].,1,NoOp(incoming call from ${CALLERID(num)})
 same => n,GotoIf($["${DB(dvsphone/open)}" = "1" & "${DB(dvsphone/active)}" = "dvsvoipms"]?open)
 same => n,Busy(20)
 same => n,Hangup()
 same => n(open),NoOp()
 same => n,GotoIf($["${CALLERID(num):-10}" = "5553456789"]?connect)
 same => n,Answer()
 same => n,Wait(1)
 same => n,Set(TRIES=0)
 same => n(ask),Read(PIN,beep,8,,1,10)
 same => n,GotoIf($["${PIN}" = "1234"]?connect)
 same => n,Set(TRIES=$[${TRIES} + 1])
 same => n,GotoIf($[${TRIES} >= 3]?bye)
 same => n,Playback(confbridge-invalid)
 same => n,Goto(ask)
 same => n(bye),Playback(confbridge-pin-bad)
 same => n,Hangup()
 same => n(connect),Answer()
 same => n,Playback(rpt/connected)
 same => n,rpt(1002,Pv)
 same => n,Hangup()
```

STEP	WHY
`GotoIf(... open ... active ...?open)`	Answer only while the Phone tab is open and this network is the picked one; otherwise `Busy(20)`.
`GotoIf(CALLERID(num):-10 = ...?connect)`	A trusted number skips the PIN (one line per number).
`Read(PIN,beep,8,,1,10)`	Beep, then up to 8 digits ending in #; three tries.
`rpt(1002,Pv)`	Join the network's node in phone mode with VOX.

NOTE With incoming set to **Hang up**, the context is just `exten => _[+0-9].,1,Hangup()` and `exten => s,1,Hangup()`.

The phone nodes in rpt.conf

RPT.CONF, BETWEEN THE DVS-PHONE MARKERS (ONE NETWORK)

```
[1001](node-main)
rxchannel = Local/pseudo
context = dvs-node-dvshoip
callerid = "Phone" <0000000000>
functions = functions1001
phone_functions = functions1001
link_functions = functions1001
duplex = 1
voxtimeout = 10000
voxrecover = 2000
simplexpatchdelay = 25
simplexphonedelay = 25

[functions1001]
61 = autopatchup,noct=1,farenddisconnect=1,dialtime=20000,context=dvs-radio-dvshoip,quiet=1
62 = cmd,/var/lib/asterisk/dvs_phone_hangup
63 = cop,9
64 = cop,10
65 = cmd,/var/lib/asterisk/dvs_phone_hangup
99 = cop,6
980 = cmd,/var/lib/asterisk/dvs_phone_tonecode 0
981 = cmd,/var/lib/asterisk/dvs_phone_tonecode 1
982 = cmd,/var/lib/asterisk/dvs_phone_tonecode 2
```

LINE	WHY
[1001](node-main)	A private node (1000–1999), built on the stock template.
rxchannel = Local/pseudo	No radio: audio only comes from links and the call.
context = dvs-node-dvshoip	Where this node's dialing starts (also where SysMon's Dialing rules card begins).
callerid = "Phone" <0000000000>	Caller ID the node presents.
functions / phone_functions / link_functions	All three use the node's own table, so your radio (over the link) and a phone caller see the same codes.
duplex = 1 + VOX lines	Half duplex with the default VOX values, so call audio passes cleanly to a simplex radio.
61 = autopatchup,...,context=dvs-radio-dvshoip,quiet=1	`*61` from the radio: collect digits, hand them to the Dashboard, end quietly.
62 / 65 = hang-up script	Hang up any phone call.
63 / 64	Phone patch on / off.
99 = cop,6	Phone-mode PTT, kept as shipped.
980 / 981 / 982	Send `*`, `#` or `*99` into the call (the Dashboard plays them).

The three helper scripts

All live in `/var/lib/asterisk`, are written by the Dashboard, and are removed by Revert.

DVS_PHONE_DIALREQ (*61 HAND-OFF)

```
#!/bin/bash
# asl_dvs_dashboard - radio dial request (written by the dashboard)
# Leaves a note for the dashboard; the dashboard places the call.
case "$1" in ''|*[!0-9]*) exit 0 ;; esac
[ ${#1} -le 20 ] || exit 0
case "$2" in ''|*[!a-z0-9]*) exit 0 ;; esac
d=/run/asl_dvs_tones/req
[ -d "$d" ] || exit 0
: > "$d/dial_$1_$2" 2>/dev/null
exit 0
```

DVS_PHONE_TONECODE (*980–*982)

```
#!/bin/bash
# asl_dvs_dashboard - radio tone code (written by the dashboard)
# Leaves a note for the dashboard; the dashboard sends the tones.
case "$1" in [0-2]) ;; *) exit 0 ;; esac
d=/run/asl_dvs_tones/req
[ -d "$d" ] || exit 0
: > "$d/code$1" 2>/dev/null
exit 0
```

DVS_PHONE_HANGUP (*62, *65)

```
#!/bin/bash
# asl_dvs_dashboard - hang up every Phone-tab call (written by the dashboard)
asterisk -rx "core show channels concise" | awk -F'!' '{print $1}' \
  | grep -E '^(IAX2|PJSIP)/dvs' | while read -r ch; do
    asterisk -rx "channel request hangup $ch"
done
```

The first two only leave an empty file named after the request in `/run/asl_dvs_tones/req` (for example `dial_5552345678_dvsvoipms`). The Dashboard checks that folder every two seconds, deletes each file and acts on its name. Nothing else is passed, so nothing else can be injected.

13 · NODE CONTROLLER

The SIP account file

`dvs_phone_pjsip.conf` defines each SIP network as a small group of PJSIP objects. All share the network's id as their name, told apart by `type=`.

DVS_PHONE_PJSIP.CONF (HAMS OVER IP PART)

```
; asl_dvs_dashboard - Phone tab SIP accounts (rewritten on every apply)

[dvs-transport]
type=transport
protocol=udp
bind=0.0.0.0:5060
local_net=192.168.1.0/24
local_net=127.0.0.1/32

; ---- HOIP ----
[dvshoip-auth]
type=auth
auth_type=userpass
username=kd8pgk
password=secret

[dvshoip-reg]
type=registration
outbound_auth=dvshoip-auth
server_uri=sip:hoip.example.net:5160
client_uri=sip:kd8pgk@hoip.example.net
retry_interval=60
contact_user=kd8pgk
expiration=3600
line=yes
endpoint=dvshoip

[dvshoip]
type=aor
contact=sip:hoip.example.net:5160
qualify_frequency=60

[dvshoip]
type=endpoint
context=dvs-in-dvshoip
disallow=all
allow=ulaw
allow=alaw
allow=gsm
aors=dvshoip
direct_media=no
rtp_symmetric=yes
force_rport=yes
rewrite_contact=yes
dtmf_mode=rfc4733
outbound_auth=dvshoip-auth
from_user=kd8pgk
from_domain=hoip.example.net

[dvshoip]
type=identify
endpoint=dvshoip
match=hoip.example.net
```

OBJECT	JOB
`type=transport`	The UDP socket on port 5060. Written only if your `pjsip.conf` has no transport; `local_net` and the public address help behind a router.
`type=auth`	Username and password for the account.
`type=registration`	Signs in so incoming calls can reach you, and tries again every 60 s if that fails (`retry_interval`). `contact_user` is the user part the provider sends your calls to. Written only for the picked network, so two accounts for one number never compete.
`type=aor`	Where to send calls (the provider's address); `qualify_frequency` checks it every 60 s.
`type=endpoint`	The account itself: codecs, NAT handling, tone mode, and `context=dvs-in-dvshoip` for incoming calls.
`type=identify`	Recognises incoming traffic from the provider's address as this account.

ENDPOINT LINE	WHY
`context=dvs-in-<net>`	Incoming calls enter the dialplan here (section 11).
`disallow=all + allow=ulaw/alaw/gsm`	Only codecs app_rpt handles well on a small Pi.
`direct_media=no`	Audio always goes through Asterisk (it must, to reach the node).
`rtp_symmetric, force_rport, rewrite_contact`	Behind-a-router (NAT) friendliness.
`dtmf_mode=rfc4733`	Keypad tones go as tone packets (RFC 2833/4733). Fixed for every SIP network since 9.3.71.5 / 9.3.73.
`callerid=`	Only when you filled in a caller ID (the VoIP.ms example has `callerid=<5550001111>`).

NOTE IAX2 networks are written to `dvs_phone_iax.conf` (one peer each, `context=dvs-in-<net>`) and `dvs_phone_iax_reg.conf` (one `register =>` line, for the picked network only).

14 · FLOWS

Call flows

A. You press Dial, a favorite, Test call or Voicemail

1. **The Dashboard checks**
   Phone tab open, patch on, a network picked and linked, number rules (911/E911, blocked kinds, international).

2. **It sends Originate**
   `Local/<number>@dvs-node-<net>/n`, Application `Rpt`, Data `<node>,Pv`, Timeout 65 s.

3. **dvs-node → dvs-net (→ -r)**
   Patch switch, saved tones, rules, then `Dial()` on the network.

4. **The far end answers**
   The Local pair answers; `rpt(<node>,Pv)` joins the call to the phone node.

5. **VOX keys the link**
   The radio transmits only while the far end talks; in the gaps your radio is heard.

B. You key *61 and a number on the radio

1. **DTMF crosses the link**
   Your radio node passes `*61` and the digits to the phone node (`link_functions`).

2. **Autopatch collects the digits**
   Unique match, or 1.5 s after the last digit; context `dvs-radio-<net>`.

3. **System() leaves a note**
   `dvs_phone_dialreq` writes `dial_<number>_<net>`; `Hangup()` ends the autopatch silently.

4. **The Dashboard dials**
   Within about 2 s, exactly as route A. The result shows as a Phone tab notice.

C. Someone calls you

1. **The provider sends the call**
   The account's endpoint (`context=dvs-in-<net>`) puts it in the dialplan.

2. **Tab open and network picked?**
   No: busy. Yes: trusted number, PIN or straight through.

3. **rpt(<node>,Pv)**
   Joined with VOX; the Phone tab shows the caller and lights a matching favorite.

D. Keypad tones during a call

SEND KEYPAD TONES	HOW THE TONES TRAVEL
Straight to the provider	The Dashboard plays them on the SIP/IAX2 leg (AMI `PlayDTMF`); SIP sends them as RFC 2833 packets.
Through the call	Played on the node side so they pass through the call like the radio's own tones; saved favorite tones go in Dial's `D()` option.
As sound	Touch-tone recordings played into the call by `dvs-tones` and ChanSpy.

E. Hanging up

YOU USE	WHAT HAPPENS
Hang up (Phone tab)	The Dashboard hangs up every PJSIP/IAX2 `dvs` channel and sends `rpt cmd <node> autopatchdn` to cancel a radio dial still collecting digits.
`*62` or `*65` **(radio)**	The hang-up script ends every phone call.
The far end hangs up	The call legs close; `rpt()` returns and the call is gone.

15 · NODE CONTROLLER

Switches and the Dashboard's part

AstDB keys

KEY	SET WHEN	READ BY
`dvsphone/open`	The Phone tab is open (1) or closed (deleted).	`dvs-in-<net>`: answer or busy.
`dvsphone/active`	You pick a network.	`dvs-in-<net>`; `dvs-phone-out`; `dvs-radio-out`.
`dvsphone/patch`	Phone patch on (1) / off (0).	`dvs-node-<net>`, `dvs-phone-out`: refuse when 0.
`dvsphone/then`	A favorite with saved tones is dialed (Through the call).	`dvs-node-<net>`: becomes Dial's `D()`; deleted on use.

WHAT IT LOOKS LIKE (EXAMPLE)

```
sudo asterisk -rx "database show dvsphone"
/dvsphone/active                                  : dvshoip
/dvsphone/open                                    : 1
/dvsphone/patch                                   : 1
```

What the Dashboard does around the dialplan

- **Save Phone**: writes the files, reloads what changed, checks each phone node is running.
- **Phone tab open**: links the picked network's node to the radio node (two-way, or monitor in Listen only), sets `open` and turns the transmitter time-out off with a safety cap.
- **Every 2 seconds**: reads the link list (`RPT_ALINKS`), fixes a link in the wrong mode, checks the request folder for `*61` dials and `*980`–`*982` tones.
- **Dial**: Originate (section 7). **Hang up**: section 14E.
- **Start-up**: compares what it would write with the files and refreshes anything stale.

Listen only

With **Listen only** ticked, the phone node is linked as a monitor link (`ilink 2` instead of `ilink 3`). app_rpt never keys or sends audio over a monitor link, so nothing from your radio, including DTMF codes, reaches the phone node. Check it with:

LINK MODE

```
sudo asterisk -rx "rpt xnode 652702" | grep ALINKS
RPT_ALINKS=1,1001RU   ; R = listen only (monitor), T = two-way; K/U = keyed/unkeyed
```

16 · COMPARE

Stock and Node Controller side by side

	STOCK ASL3	NODE CONTROLLER
Out of the box	Off: codes commented out, `pstn-out` says "no service"	Off until you Activate it on the Edit page
Where the patch lives	On your radio node	A private node per phone network, linked only while the Phone tab is open
Files	You edit `rpt.conf` and `extensions.conf`	Its own included files, a marked block, one include line
Several providers	Your own contexts	One block of contexts per network; one picked at a time
Dialing from the radio	`*61`: autopatch dials	`*61`: autopatch collects, the Dashboard dials
Dialing from a browser	—	Dial, favorites, Test call, Voicemail
How the call joins	Autopatch (link keyed all call)	`rpt(node,Pv)`: phone mode with VOX
Simplex hotspot	Dead key	Keys only while the far end talks
Blocked numbers	`check_route`	Same kinds, per network (`-r` context)
911	Blocked: `check_route` sends `_X11` to "no service"	Only with "E911 is set up"
Incoming	Your own (examples: `iax-client`, `allstar-sys`)	Only while the tab is open; PIN or trusted numbers
Tones	Radio DTMF only	Keypad, favorites, `*980`–`*982`; three routes
Listen only	—	Monitor link option
Ringing on the air	Yes	No: you hear the call once answered
Seeing it	Asterisk CLI	Asterisk CLI, SysMon Phone tab, Dashboard report

WHY Both use the same building blocks: contexts, patterns, `Dial()` and `rpt()`. Node Controller writes more of them, keeps them in its own file, and moves the call from autopatch to phone mode so a simplex radio can talk back.

CAREFUL Trade-offs of phone mode: no ringback on the air; about half a second of audio delay; after 10 s of nonstop far-end talk the radio unkeys for 2 s (`voxtimeout` / `voxrecover`); and radio-dialed calls need the Phone tab open, which is also the only time your radio is linked to the phone node.

Watching a call and fixing problems

Watch a call step by step

CONSOLE

```
sudo asterisk -rvvv   ; open the console with more detail (Ctrl-C to leave)
core set verbose 3    ; or set it inside a running console
```

Each step the dialplan runs is printed as `Executing [number@context:step] App(...)`. A Dashboard call to VoIP.ms (555-234-5678) looks like this (trimmed, illustrative):

A CALL IN THE LOG

```
-- Executing [5552345678@dvs-node-dvsvoipms:1] Set("Local/5552345678@dvs-node-dvsvoipms-00000004;2",
"DVSNET=dvsvoipms")
-- Executing [5552345678@dvs-node-dvsvoipms:6] Goto("Local/...;2", "dvs-net-dvsvoipms,5552345678,1")
-- Executing [5552345678@dvs-net-dvsvoipms:1] Goto("Local/...;2", "dvs-net-dvsvoipms-
r,15552345678,1")
-- Executing [15552345678@dvs-net-dvsvoipms-r:3] Dial("Local/...;2",
"PJSIP/15552345678@dvsvoipms,60,")
-- Called PJSIP/15552345678@dvsvoipms
-- PJSIP/dvsvoipms-0000000a answered Local/...;2
-- Executing [...] Rpt("Local/...;1", "1002,Pv")
```

Useful commands

COMMAND	SHOWS
`sudo asterisk -rx "dialplan show dvs-net-dvsvoipms"`	The loaded rules for one context.
`sudo asterisk -rx "dialplan show 15552345678@dvs-net-dvsvoipms-r"`	Which line a number would match.
`sudo asterisk -rx "dialplan reload"`	Re-read `extensions.conf` and its includes.
`sudo asterisk -rx "database show dvsphone"`	The switches.
`sudo asterisk -rx "core show channels concise"`	Live call legs.
`sudo asterisk -rx "rpt xnode 652702" | grep -E "ALINKS|TXKEYED"`	Link mode and whether the radio is keyed.
`sudo asterisk -rx "pjsip show endpoint dvsvoipms"`	One SIP account, including `dtmf_mode`.
`sudo asterisk -rx "pjsip show registrations"`	Sign-in state (the picked network only).
`sudo journalctl -u nodecontrol -n 50 | grep -i phone`	The Dashboard's phone log (dials, radio codes, link fixes).

Common problems

YOU SEE	LIKELY CAUSE	CHECK OR FIX
"No service" on every call	Patch switched off, or the number fits no rule	`database show dvsphone` (patch = 1?); `dialplan show <number>@dvs-net-<net>`

"Dial a 10-digit number" for a real-looking number	The area code or exchange starts with 0 or 1	Check the number; US/Canada area codes and exchanges start with 2–9
Incoming callers get busy	Phone tab closed, or another network picked	Open the tab and pick that network
Radio `*61` does nothing	Tab closed, Listen only on, or `app_system` not loaded	`module show like app_system`; the Dashboard log
Radio keyed for the whole call	An old autopatch call (before 9.3.71.8 / 9.3.73)	Update; check `TXKEYED` drops to 0 in silence
Far end can't hear you	Hotspot can't hear you, or Listen only	`TXKEYED` = 0 in the gaps? `ALINKS` shows T, not R?
Tones not recognised	Provider wants another tone route	Try the other **Send keypad tones** choices
Your own context vanished	It was inside a `dvs` file or the markers	Keep your edits outside them

Quick reference

RADIO CODE	DOES
`*61<number>`	Dial (through the Dashboard)
`*62 / *65`	Hang up
`*63 / *64`	Patch on / off
`*980`	Send *
`*981`	Send #
`*982`	Send *99
`*99`	Phone-mode PTT (stock)

SPECIAL NUMBER	MEANS
`0098`	Test call
`0097`	Voicemail
`911`	Only with E911 set up
`s`	Call with no number
`i`	Matched nothing
`t`	Timed out

PATTERN CHAR	MATCHES
`X`	0–9
`Z`	1–9
`N`	2–9
`[...]`	one of the set
`.`	one or more of anything
`!`	zero or more, match at once

RPT() OPTION	MEANS
`P`	phone control
`D`	dumb phone (keyed all call)
`S`	dumb simplex (* toggles)
`R`	reverse autopatch
`X`	link without security check
`v`	add VOX
`m`	monitor only

FILE	OWNER	RELOAD
extensions.conf	you (+1 include line)	`dialplan reload`
dvs_phone_extensions.conf	Dashboard	`dialplan reload`
rpt.conf	you (+ marked block)	`module reload app_rpt`
dvs_phone_pjsip.conf	Dashboard	`module reload res_pjsip.so`
dvs_phone_iax*.conf	Dashboard	`iax2 reload`
/var/lib/asterisk/dvs_phone_*	Dashboard	—

Sources: AllStarLink app_rpt at master `d392975` (sample configs, `app_rpt.c`, `rpt_functions.c`, `rpt_link.c`, `rpt_vox.c`), Asterisk dialplan documentation, and the Node Controller Dashboard and SysMon code named on the cover. The Node Controller excerpts were generated from that Dashboard code; the source of this guide is in `docs/src/dialplan_guide/`.